

JavaScript Code

Arena Strike 3D - Poora game logic: player movement, shooting, enemy AI, loot system, wave spawning, touch controls, collision detection, HUD updates. (Ismein Three.js library bhi call hoti hai - uske specific parts alag PDF mein bhi diye hain.)

```
(function(){
  const isTouch = ('ontouchstart' in window) || navigator.maxTouchPoints > 0;
  document.getElementById('mobileHint').style.display = isTouch ? 'block' : 'none';
  document.getElementById('desktopHint').style.display = isTouch ? 'none' : 'block';

  // ----- basic setup -----
  const scene = new THREE.Scene();
  scene.background = new THREE.Color(0x0b1622);
  scene.fog = new THREE.Fog(0x0b1622, 20, 75);

  const camera = new THREE.PerspectiveCamera(75, window.innerWidth/window.innerHeight, 0.1, 1000);
  camera.position.set(0, 1.7, 0);

  const renderer = new THREE.WebGLRenderer({antialias:true});
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.setPixelRatio(Math.min(window.devicePixelRatio,2));
  renderer.shadowMap.enabled = true;
  document.body.appendChild(renderer.domElement);

  window.addEventListener('resize', ()=>{
    camera.aspect = window.innerWidth/window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
  });

  // ----- lights -----
  scene.add(new THREE.AmbientLight(0x8899aa, 0.7));
  const sun = new THREE.DirectionalLight(0xffffff, 0.9);
  sun.position.set(30,40,10);
  sun.castShadow = true;
  sun.shadow.mapSize.set(1024,1024);
  sun.shadow.camera.left=-40; sun.shadow.camera.right=40;
  sun.shadow.camera.top=40; sun.shadow.camera.bottom=-40;
  scene.add(sun);

  // ----- arena -----
  const ARENA = 34;
  const groundGeo = new THREE.PlaneGeometry(ARENA*2, ARENA*2, 10, 10);
  const groundMat = new THREE.MeshStandardMaterial({color:0x2c3e40, roughness:0.95});
  const ground = new THREE.Mesh(groundGeo, groundMat);
  ground.rotation.x = -Math.PI/2;
  ground.receiveShadow = true;
  scene.add(ground);

  const wallMat = new THREE.MeshStandardMaterial({color:0x1a2530, roughness:0.9});
  function addWall(x,z,w,d,h){
    const wall = new THREE.Mesh(new THREE.BoxGeometry(w,h,d), wallMat);
    wall.position.set(x,h/2,z);
    wall.castShadow = true; wall.receiveShadow = true;
    scene.add(wall);
  }
  addWall(0,-ARENA,ARENA*2,1,6);
  addWall(0,ARENA,ARENA*2,1,6);
  addWall(-ARENA,0,1,ARENA*2,6);
  addWall(ARENA,0,1,ARENA*2,6);

  const coverMat = new THREE.MeshStandardMaterial({color:0x4a5a52, roughness:0.85});
  const obstacles = [];
  function addBox(x,z,sx,sy,sz){
    const b = new THREE.Mesh(new THREE.BoxGeometry(sx,sy,sz), coverMat);
    b.position.set(x, sy/2, z);
    b.castShadow = true; b.receiveShadow = true;
    scene.add(b);
    obstacles.push({x:x, z:z, hw:sx/2, hd:sz/2});
  }
  const layout = [
    [8,8,4,2,2,4],[ -8,8,3,2,6,3],[8, -8,4,2,4],[ -8, -8,3,2,4,5],
    [0,14,6,2,2],[0, -14,6,2,2],[14,0,2,2,6],[ -14,0,2,2,6],
```

```

    [4,0,2,1.6,2],[-5,3,2,1.8,2],[18,18,3,2,3],[-18,-18,3,2,3]
];
layout.forEach(a=>addBox(...a));

// ----- weapons -----
const WEAPONS = {
  Pistol: {name:'Pistol', damage:34, magSize:30, reserveMax:90, cooldown:180, spread:0.01,
    color:0xfffff},
  SMG: {name:'SMG', damage:18, magSize:40, reserveMax:160,cooldown:85, spread:0.03,
    color:0x3fa9f5},
  Shotgun: {name:'Shotgun', damage:16, magSize:8, reserveMax:32, pellets:6, cooldown:600,
    spread:0.09, color:0xf5a623},
  Sniper: {name:'Sniper', damage:100,magSize:5, reserveMax:20, cooldown:950, spread:0.002,
    color:0xd0021b}
};
let currentWeaponKey = 'Pistol';
let ammo = WEAPONS.Pistol.magSize;
let reserve = { Pistol:90, SMG:0, Shotgun:0, Sniper:0 };
let reloading = false;
let lastShotTime = 0;

// ----- player state -----
const player = { height:1.7, speed:5.2, sprintMult:1.6, health:100, maxHealth:100,
  pos:new THREE.Vector3(0,0,0), yaw:0, pitch:0, alive:true };

let score = 0, wave = 1, kills = 0, killsNeededForWave = 5;

// ----- pointer lock (desktop) / tap-start (mobile) -----
const canvas = renderer.domElement;
let gameActive = false;
const overlay = document.getElementById('overlay');
const startBtn = document.getElementById('startBtn');
const touchControls = document.getElementById('touchControls');

function beginGame(){
  if(isTouch){
    gameActive = true;
    overlay.classList.add('hidden');
    touchControls.classList.add('active');
  } else {
    canvas.requestPointerLock();
  }
}
startBtn.addEventListener('click', beginGame);

document.addEventListener('pointerlockchange', ()=>{
  if(isTouch) return;
  gameActive = (document.pointerLockElement === canvas);
  overlay.classList.toggle('hidden', gameActive);
  if(!gameActive && player.alive){
    overlay.classList.remove('hidden');
    document.getElementById('finalStats').style.display='none';
    startBtn.style.display='inline-block';
    startBtn.textContent = 'Resume (Click)';
  }
});

document.addEventListener('mousemove', (e)=>{
  if(!gameActive || isTouch) return;
  const sens = 0.0022;
  player.yaw -= e.movementX * sens;
  player.pitch -= e.movementY * sens;
  player.pitch = Math.max(-Math.PI/2+0.05, Math.min(Math.PI/2-0.05, player.pitch));
});

// ----- keyboard (desktop) -----
const keys = {};
window.addEventListener('keydown', e=>{
  keys[e.code]=true;
  if(e.code==='KeyR') tryReload();
  if(e.code==='Digit1') switchWeapon('Pistol');
  if(e.code==='Digit2') switchWeapon('SMG');
  if(e.code==='Digit3') switchWeapon('Shotgun');
  if(e.code==='Digit4') switchWeapon('Sniper');
});
window.addEventListener('keyup', e=> keys[e.code]=false);

// ----- mobile joystick -----
let joyVec={x:0,y:0};

```

```

const joyBase = document.getElementById('joyBase');
const joyStick = document.getElementById('joyStick');
let joyTouchId = null, joyOrigin={x:0,y:0};

joyBase.addEventListener('touchstart', e=>{
  e.preventDefault();
  const t = e.changedTouches[0];
  joyTouchId = t.identifier;
  const rect = joyBase.getBoundingClientRect();
  joyOrigin = {x: rect.left+rect.width/2, y: rect.top+rect.height/2};
}, {passive:false});

window.addEventListener('touchmove', e=>{
  for(const t of e.changedTouches){
    if(t.identifier===joyTouchId){
      let dx = t.clientX - joyOrigin.x, dy = t.clientY - joyOrigin.y;
      const max = 42;
      const dist = Math.min(Math.hypot(dx,dy), max);
      const ang = Math.atan2(dy,dx);
      dx = Math.cos(ang)*dist; dy = Math.sin(ang)*dist;
      joyStick.style.transform = `translate(${dx}px,${dy}px)`;
      joyVec = {x: dx/max, y: dy/max};
    }
  }
}, {passive:false});

window.addEventListener('touchend', e=>{
  for(const t of e.changedTouches){
    if(t.identifier===joyTouchId){
      joyTouchId=null; joyVec={x:0,y:0};
      joyStick.style.transform = 'translate(0,0)';
    }
  }
});

// ----- mobile look (drag on right zone) -----
const lookZone = document.getElementById('lookZone');
let lookTouchId=null, lastLookX=0, lastLookY=0;
lookZone.addEventListener('touchstart', e=>{
  e.preventDefault();
  const t = e.changedTouches[0];
  lookTouchId = t.identifier;
  lastLookX = t.clientX; lastLookY = t.clientY;
}, {passive:false});
lookZone.addEventListener('touchmove', e=>{
  e.preventDefault();
  for(const t of e.changedTouches){
    if(t.identifier===lookTouchId){
      const dx = t.clientX-lastLookX, dy = t.clientY-lastLookY;
      lastLookX=t.clientX; lastLookY=t.clientY;
      const sens = 0.0032;
      player.yaw -= dx*sens;
      player.pitch -= dy*sens;
      player.pitch = Math.max(-Math.PI/2+0.05, Math.min(Math.PI/2-0.05, player.pitch));
    }
  }
}, {passive:false});
lookZone.addEventListener('touchend', e=>{
  for(const t of e.changedTouches) if(t.identifier===lookTouchId) lookTouchId=null;
});

// ----- fire / reload buttons (mobile) -----
document.getElementById('fireBtn').addEventListener('touchstart', e=>{
  e.preventDefault(); shoot();
}, {passive:false});
document.getElementById('reloadBtnMobile').addEventListener('touchstart', e=>{
  e.preventDefault(); tryReload();
}, {passive:false});

// ----- shooting (desktop click) -----
const raycaster = new THREE.Raycaster();
const muzzleEl = document.getElementById('muzzle');
const hitFlashEl = document.getElementById('hitFlash');

canvas.addEventListener('mousedown', (e)=>{
  if(!gameActive || isTouch || e.button!==0) return;
  shoot();
});
function switchWeapon(key){

```

```

    if(currentWeaponKey===key || reloading) return;
    currentWeaponKey = key;
    ammo = WEAPONS[key].magSize;
    updateHUD();
}

function tryReload(){
    if(reloading || !player.alive) return;
    const w = WEAPONS[currentWeaponKey];
    if(ammo===w.magSize || reserve[currentWeaponKey]<=0) return;
    reloading = true;
    document.getElementById('reloadHint').textContent = 'Reloading...';
    setTimeout(()=>{
        const need = w.magSize - ammo;
        const take = Math.min(need, reserve[currentWeaponKey]);
        ammo += take;
        reserve[currentWeaponKey] -= take;
        reloading = false;
        updateHUD();
    }, 1000);
}

function shoot(){
    if(!gameActive || !player.alive || reloading) return;
    const w = WEAPONS[currentWeaponKey];
    const now = performance.now();
    if(now - lastShotTime < w.cooldown) return;
    if(ammo<=0){ tryReload(); return; }
    lastShotTime = now;
    ammo--;
    updateHUD();
    muzzleEl.style.opacity = 1;
    setTimeout(()=>muzzleEl.style.opacity=0, 60);

    const pellets = w.pellets || 1;
    const meshes = enemies.map(en=>en.mesh);
    for(let p=0;p<pellets;p++){
        const sx = (Math.random()-0.5)*w.spread*2;
        const sy = (Math.random()-0.5)*w.spread*2;
        raycaster.setFromCamera({x:sx,y:sy}, camera);
        const hits = raycaster.intersectObjects(meshes, true);
        if(hits.length>0){
            let obj = hits[0].object;
            while(obj.parent && !obj.userData.enemyRef) obj = obj.parent;
            const enemy = obj.userData.enemyRef;
            if(enemy) damageEnemy(enemy, w.damage);
        }
    }
}

// ----- enemies -----
const enemies = [];
const enemyGeo = new THREE.CapsuleGeometry(0.45, 1.1, 4, 8);
function spawnEnemy(){
    const mat = new THREE.MeshStandardMaterial({color:0xc0392b, roughness:0.7});
    const mesh = new THREE.Mesh(enemyGeo, mat);
    mesh.castShadow = true;
    let x,z;
    do{
        const ang = Math.random()*Math.PI*2;
        const r = 15 + Math.random()*15;
        x = Math.cos(ang)*r; z = Math.sin(ang)*r;
    } while(Math.abs(x)>ARENA-2 || Math.abs(z)>ARENA-2);
    mesh.position.set(x, 1.05, z);
    scene.add(mesh);
    const enemy = { mesh, health:60, speed: 1.3+Math.random()*0.6+wave*0.06, alive:true, attackCd:0 };
    mesh.userData.enemyRef = enemy;
    enemies.push(enemy);
}

function damageEnemy(enemy, dmg){
    if(!enemy.alive) return;
    enemy.health -= dmg;
    enemy.mesh.material.emissive = new THREE.Color(0x330000);
    setTimeout(()=>{ if(enemy.mesh.material.emissive = new THREE.Color(0x000000); },80);
    if(enemy.health<=0){
        enemy.alive = false;
    }
}

```

```

        scene.remove(enemy.mesh);
        score += 100;
        kills++;
        updateHUD();
        checkWave();
    }
}

function checkWave(){
    if(kills >= killsNeededForWave){
        wave++; kills = 0; killsNeededForWave = 5 + wave*2;
        document.getElementById('waveVal').textContent = wave;
        const toSpawn = 3 + wave;
        for(let i=0;i<toSpawn;i++) setTimeout(spawnEnemy, i*250);
    }
}

for(let i=0;i<4;i++) spawnEnemy();

// ----- loot system -----
const LOOT_TYPES = ['ammo', 'health', 'weapon'];
const lootPoints = [
    [10, 3], [-10, 3], [3, 10], [3, -10], [-6, -11], [6, 11], [-16, 6], [16, -6],
    [0, 0], [20, -20], [-20, 20], [12, -14]
];
const loot = [];
function spawnLootAt(idx){
    const [x, z] = lootPoints[idx];
    const type = LOOT_TYPES[Math.floor(Math.random()*LOOT_TYPES.length)];
    let color, weaponKey=null;
    if(type==='ammo') color=0xfffd23f;
    else if(type==='health') color=0x4caf50;
    else { const keys=['SMG', 'Shotgun', 'Sniper']; weaponKey =
        keys[Math.floor(Math.random()*keys.length)]; color = WEAPONS[weaponKey].color; }
    const geo = new THREE.OctahedronGeometry(0.4, 0);
    const mat = new THREE.MeshStandardMaterial({color, emissive:color, emissiveIntensity:0.35,
        roughness:0.4});
    const mesh = new THREE.Mesh(geo, mat);
    mesh.position.set(x, 1.1, z);
    mesh.castShadow = true;
    scene.add(mesh);
    loot[idx] = { mesh, x, z, type, weaponKey, active:true, bob: Math.random()*Math.PI*2 };
}
lootPoints.forEach((_, i) => spawnLootAt(i));

function showPickupToast(text){
    const el = document.getElementById('pickupToast');
    el.textContent = text;
    el.style.opacity = 1;
    el.style.transform = 'translate(-50%, -60%)';
    setTimeout(() => { el.style.opacity=0; el.style.transform='translate(-50%, -50%)'; }, 700);
}

function collectLoot(item){
    item.active = false;
    scene.remove(item.mesh);
    if(item.type==='ammo'){
        const w = WEAPONS[currentWeaponKey];
        reserve[currentWeaponKey] = Math.min(w.reserveMax, reserve[currentWeaponKey] +
            Math.round(w.magSize*1.5));
        showPickupToast('+ Ammo');
    } else if(item.type==='health'){
        player.health = Math.min(player.maxHealth, player.health + 35);
        showPickupToast('+ Health');
    } else {
        currentWeaponKey = item.weaponKey;
        reserve[item.weaponKey] = Math.max(reserve[item.weaponKey],
            WEAPONS[item.weaponKey].magSize*2);
        ammo = WEAPONS[item.weaponKey].magSize;
        showPickupToast('Picked up ' + item.weaponKey);
    }
    updateHUD();
    const idx = loot.indexOf(item);
    setTimeout(() => { if(idx >= 0) spawnLootAt(idx); }, 14000);
}

// ----- collision helper -----
function collidesWithObstacle(x, z, radius){
    for(const o of obstacles){
        if(Math.abs(x-o.x) < o.hw+radius && Math.abs(z-o.z) < o.hd+radius) return true;
    }
}

```

```

    }
    return false;
}

function flashDamage(){
    hitFlashEl.style.opacity = 0.55;
    setTimeout(()=>hitFlashEl.style.opacity=0, 150);
}

// ----- HUD -----
function updateHUD(){
    document.getElementById('scoreVal').textContent = score;
    document.getElementById('weaponName').textContent = currentWeaponKey;
    document.getElementById('ammoVal').textContent = ammo;
    document.getElementById('reserveVal').textContent = reserve[currentWeaponKey];
    document.getElementById('healthBarInner').style.width = Math.max(0,player.health) + '%';
    if(!reloading) document.getElementById('reloadHint').textContent = isTouch ? 'Tap reload' : 'R
        to reload';
}
updateHUD();

// ----- minimap -----
const mmCanvas = document.getElementById('minimap');
const mmCtx = mmCanvas.getContext('2d');
function drawMinimap(){
    const S = 130, scale = S/(ARENA*2.2);
    mmCtx.clearRect(0,0,S,S);
    mmCtx.fillStyle = 'rgba(20,30,35,0.6)';
    mmCtx.fillRect(0,0,S,S);
    function toMM(x,z){ return [S/2 + x*scale, S/2 + z*scale]; }
    mmCtx.fillStyle = 'rgba(120,140,130,0.5)';
    obstacles.forEach(o=>{
        const [mx,mz] = toMM(o.x,o.z);
        mmCtx.fillRect(mx-o.hw*scale, mz-o.hd*scale, o.hw*2*scale, o.hd*2*scale);
    });
    loot.forEach(item=>{
        if(!item || !item.active) return;
        const [mx,mz] = toMM(item.x,item.z);
        mmCtx.fillStyle = item.type==='ammo' ? '#ffd23f' : item.type==='health' ? '#4caf50' :
            '#3fa9f5';
        mmCtx.beginPath(); mmCtx.arc(mx,mz,3,0,7); mmCtx.fill();
    });
    mmCtx.fillStyle = '#e74c3c';
    enemies.forEach(en=>{
        if(!en.alive) return;
        const [mx,mz] = toMM(en.mesh.position.x, en.mesh.position.z);
        mmCtx.beginPath(); mmCtx.arc(mx,mz,3,0,7); mmCtx.fill();
    });
    const [px,pz] = toMM(player.pos.x, player.pos.z);
    mmCtx.save();
    mmCtx.translate(px,pz);
    mmCtx.rotate(player.yaw);
    mmCtx.fillStyle = '#ffb020';
    mmCtx.beginPath();
    mmCtx.moveTo(0,-6); mmCtx.lineTo(4,5); mmCtx.lineTo(-4,5); mmCtx.closePath(); mmCtx.fill();
    mmCtx.restore();
}

// ----- game over -----
function gameOver(){
    player.alive = false;
    if(!isTouch) document.exitPointerLock();
    gameActive = false;
    touchControls.classList.remove('active');
    overlay.classList.remove('hidden');
    startBtn.style.display='none';
    const stats = document.getElementById('finalStats');
    stats.style.display='block';
    stats.innerHTML = 'GAME OVER<br>Final Score: '+score+' &nbsp;&nbsp;  Wave Reached: '+wave;
    overlay.querySelector('h1').textContent = 'Defeated';
}

// ----- main loop -----
const clock = new THREE.Clock();
function animate(){
    requestAnimationFrame(animate);
    const dt = Math.min(clock.getDelta(), 0.05);

    if(gameActive && player.alive){

```

```

const forward = new THREE.Vector3(Math.sin(player.yaw), 0, Math.cos(player.yaw)).negate();
const right = new THREE.Vector3(Math.cos(player.yaw), 0, -Math.sin(player.yaw));
let move = new THREE.Vector3();

if(isTouch){
  if(Math.abs(joyVec.x)>0.08 || Math.abs(joyVec.y)>0.08){
    move.add(forward.clone().multiplyScalar(-joyVec.y));
    move.add(right.clone().multiplyScalar(joyVec.x));
  }
} else {
  if(keys['KeyW']) move.add(forward);
  if(keys['KeyS']) move.sub(forward);
  if(keys['KeyD']) move.add(right);
  if(keys['KeyA']) move.sub(right);
}

if(move.lengthSq(>0)){
  move.normalize();
  const spd = player.speed * ((!isTouch && (keys['ShiftLeft']||keys['ShiftRight'])) ?
    player.sprintMult : 1);
  const nx = player.pos.x + move.x*spd*dt;
  const nz = player.pos.z + move.z*spd*dt;
  const clampedX = Math.max(-ARENA+1, Math.min(ARENA-1, nx));
  const clampedZ = Math.max(-ARENA+1, Math.min(ARENA-1, nz));
  if(!collidesWithObstacle(clampedX, player.pos.z, 0.5)) player.pos.x = clampedX;
  if(!collidesWithObstacle(player.pos.x, clampedZ, 0.5)) player.pos.z = clampedZ;
}
camera.position.set(player.pos.x, player.height, player.pos.z);
const dir = new THREE.Vector3(
  Math.sin(player.yaw)*Math.cos(player.pitch),
  Math.sin(player.pitch),
  -Math.cos(player.yaw)*Math.cos(player.pitch)
);
camera.lookAt(camera.position.clone().add(dir));

enemies.forEach(en=>{
  if(!en.alive) return;
  const ex=en.mesh.position.x, ez=en.mesh.position.z;
  const dx = player.pos.x - ex, dz = player.pos.z - ez;
  const dist = Math.sqrt(dx*dx+dz*dz);
  if(dist > 1.6){
    const nx = ex + (dx/dist)*en.speed*dt;
    const nz = ez + (dz/dist)*en.speed*dt;
    en.mesh.position.x = nx;
    en.mesh.position.z = nz;
    en.mesh.rotation.y = Math.atan2(dx,dz);
  } else {
    en.attackCd -= dt;
    if(en.attackCd<=0){
      en.attackCd = 0.8;
      player.health -= 8;
      flashDamage();
      updateHUD();
      if(player.health<=0) gameOver();
    }
  }
});

loot.forEach(item=>{
  if(!item || !item.active) return;
  item.bob += dt*2.2;
  item.mesh.position.y = 1.1 + Math.sin(item.bob)*0.12;
  item.mesh.rotation.y += dt*1.4;
  const dx = player.pos.x-item.x, dz = player.pos.z-item.z;
  if(Math.sqrt(dx*dx+dz*dz) < 1.4) collectLoot(item);
});

drawMinimap();
}

renderer.render(scene, camera);
}
animate();
})();

```